



Database Management & Reporting

Online Bookstore Management System

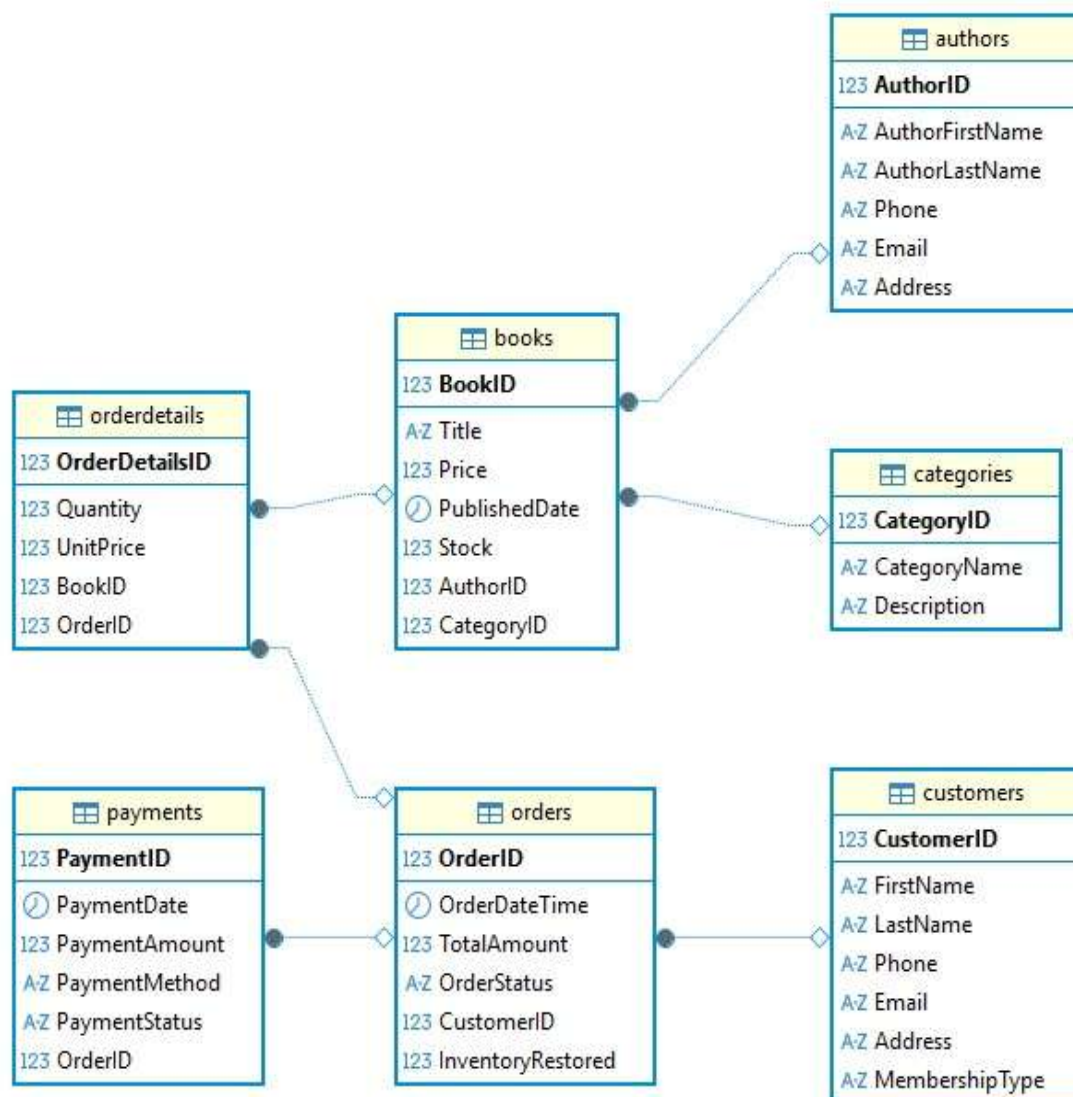
Submitted by

Jui Barua (N01772035)

1. Entity-Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) shows how the bookstore database is set up overall. It includes the main parts of the system, like books, authors, customers, orders, and payments, and shows how they are connected to each other. By looking at the ERD, it becomes easier to understand how the data is organized and how different parts of the system interact. This helps make sure the database is structured in a clear and logical way so that storing and retrieving information works efficiently.

1.1 ERD Diagram



1.2 Table Relationships Description

This section explains how the different tables in the database are connected. It shows how each entity links to others using primary and foreign keys. These connections help keep the data consistent and organized, while also making it easier for information to move across the system and support everyday operations.

Parent Table	Child Table	Relationship Type	Foreign Key in Child	Description
authors	books	One-to-Many (1:N)	AuthorID	One author can write multiple books, but each book is written by one author
categories	books	One-to-Many (1:N)	CategoryID	One category can include many books, but each book belongs to one category.
books	orderdetails	One-to-Many (1:N)	BookID	A book can appear in many order details, but each order detail refers to one book.
orders	orderdetails	One-to-Many (1:N)	OrderID	One order can contain multiple order details (items), but each order detail belongs to one order.
customers	orders	One-to-Many (1:N)	CustomerID	A customer can place multiple orders, but each order is placed by one customer.
orders	payments	One-to-Many (1:N)	OrderID	One order can have multiple payments (or payment attempts), but each payment is linked to one order.

2. Data Dictionary

The data dictionary provides detailed information about the database structure, including tables, columns, data types, and descriptions. It helps ensure consistency, improves understanding of the data, and serves as a reference for database design and usage.

2.1 Customers Table

Field Name	Data Type	Key	Constraints / Description
CustomerID	INT	PK	Auto-increment, unique identifier
FirstName	VARCHAR (50)		Customer's first name
LastName	VARCHAR (50)		Customer's last name
Phone	VARCHAR (15)		Optional (NULL allowed)
Email	VARCHAR (50)		Unique email address
Address	VARCHAR (100)		Customer's address
MembershipType	VARCHAR (20)		Membership level (Default: 'Regular')

2.2 Authors Table

Field Name	Data Type	Key	Description
AuthorID	INT	PK	Auto-increment, unique identifier
AuthorFirstName	VARCHAR (50)		Author's first name
AuthorLastName	VARCHAR (50)		Author's last name
Phone	VARCHAR (15)		Optional
Email	VARCHAR (50)		Unique email
Address	VARCHAR (100)		Author's address

2.3 Categories Table

Field Name	Data Type	Key	Description
CategoryID	INT	PK	Auto-increment, unique identifier
CategoryName	VARCHAR (50)		Name of category
Description	VARCHAR (100)		Category Details

2.4 Books Table

Field Name	Data Type	Key	Description
BookID	INT	PK	Auto-increment, unique identifier
Title	VARCHAR (100)		Book title
Price	DECIMAL (6,2)		Price of the book
PublishedDate	DATE		Date of publication
Stock	INT		Available quantity
AuthorID	INT	FK	References Authors (AuthorID)
CategoryID	INT	FK	References Categories (CategoryID)

2.5 Orders Table

Field Name	Data Type	Key	Description
OrderID	INT	PK	Auto-increment, unique identifier
OrderDateTime	DATETIME		Date and time of order
TotalAmount	DECIMAL (8,2)		Total order amount
OrderStatus	VARCHAR (15)		Status (Paid, Pending, Cancelled, Shipped)
CustomerID	INT	FK	References Customers (CustomerID)

InventoryRestored	BOOLEAN		Indicates if stock has been restored (Default: FALSE)
-------------------	---------	--	---

2.6 OrderDetails Table

Field Name	Data Type	Key	Description
OrderDetailsID	INT	PK	Auto-increment, unique identifier
Quantity	INT		Number of books ordered
UnitPrice	DECIMAL (6,2)		Price per book at time of purchase
BookID	INT	FK	References Books (BookID)
OrderID	INT	FK	References Orders (OrderID)

2.7 Payments Table

Field Name	Data Type	Key	Description
PaymentID	INT	PK	Auto-increment, unique identifier
PaymentDate	DATE		Date of payment
PaymentAmount	DECIMAL (8,2)		Amount paid
PaymentMethod	VARCHAR (15)		Payment type (Credit Card, Debit Card, PayPal, Cash)
PaymentStatus	VARCHAR (15)		Status (Completed, Pending, Failed)
OrderID	INT	FK	References Orders (OrderID)

3. Detailed Descriptions

3.1 Views

3.1.1 vw_available_books_by_category

This view basically works as a live digital catalog. Instead of making us jump between three different tables to find information, it pulls everything into one spot, so we can see exactly what's on the shelves. It gathers the book ID, title, price, and stock levels, but it also pulls in the category name and author details, so the data makes sense to a person, not just a computer.

To keep things clean, it merges the author's first and last names into a single "Author Name" column. The most important part, though, is the filter: it only shows books where the stock is greater than zero. This is perfect for a "Customer View" because it automatically hides anything that's sold out, making it much easier to manage sales or browsing without extra work.

3.1.2 vw_customer_order_summary

The vw_customer_order_summary view was designed to make tracking sales a lot more straightforward. Basically, it pulls data from the **Orders** and **Customers** tables and puts it all in one spot, so there is no need to manually jump back and forth between different parts of the database.

It covers all the essentials: the order ID, the date and time of the purchase, the total cost, and the current status of the order. The customer's first and last names are combined into a single column. Everything is linked together using the CustomerID, which ensures that every order is accurately matched to the person who made it.

Since there aren't any filters on this view, it shows the entire order history regardless of whether a sale is completed or still pending. This makes it a great resource for looking back at past activities or just getting a general sense of how things are moving. Overall, it's a huge time-saver because it handles all the messy background connections and gives a clean, organized look at the business at any given moment.

3.1.3 vw_revenue_by_category

To get a better sense of which genres or sections are driving the most business, the vw_revenue_by_category view breaks performance by category. It connects to the **OrderDetails**,

Books, Categories, and Orders tables to give an accurate picture of sales volume and total income.

The logic here focuses on two main metrics: the total number of books sold per category and the total revenue (calculated by multiplying quantity by price). To ensure the numbers stay accurate for financial reporting, the view only includes orders that are marked as "Paid" or "Shipped." This prevents pending or cancelled orders from skewing the results. By grouping everything by category, the view provides a clear summary of which areas are the most profitable, making it a key resource for analyzing business growth and inventory success.

3.2 User-Defined Functions (UDFs)

3.2.1 fn_get_discount_rate

The **user-defined function (fn_get_discount_rate)** calculates the **discount percentage based on a customer's membership type**. The function takes a **membership value as input** and returns a **numeric discount rate (decimal)**. It uses a **CASE statement** to apply predefined business rules, assigning **5% discount for Silver, 10% for Gold, and 15% for Platinum**, while any other membership type receives **0% discount**. The use of the **UPPER()** function ensures that the comparison is **case-insensitive**, allowing flexibility in input values. The function is marked as **DETERMINISTIC**, meaning it will always return the **same output for the same input**, ensuring consistency. Overall, this function centralizes the **discount logic**, making it **reusable and consistent**.

To verify the functionality, the following test can be executed:

```
-- Test the function by passing a membership type
SELECT fn_get_discount_rate('Gold'); -- output: 10.00
```

This test confirms that a **Gold membership correctly returns a 10% discount**.

3.2.2 fn_discounted_price

The **user-defined function (fn_discounted_price)** calculates the **final price of a product after applying a discount**. The function takes two input parameters: the **original price**

(**p_original_price**) and the **discount rate (p_discount_rate)**, both defined as **decimal values for precise monetary calculations**. It returns a **decimal value representing the discounted price**. The function is marked as **DETERMINISTIC**, meaning it will always produce the **same output for the same input values**, ensuring **consistency and optimization** in database operations. Inside the function, a variable named **final_price** is declared to store the computed result. The calculation is performed using a **standard percentage-based formula**:
Final Price = Original Price - (Original Price × Discount Rate / 100). Here, the **discount amount is calculated first and then subtracted from the original price** to obtain the final price. Once the calculation is complete, the function **returns the computed value**.

Overall, this function encapsulates the **pricing logic**, making it **reusable, consistent, and easy to integrate** into queries and stored procedures. It can be effectively combined with **fn_get_discount_rate** to dynamically apply discounts based on **customer membership type**.

To verify the functionality, the following test can be executed:

```
-- Test the function
SELECT fn_discounted_price(1000, fn_get_discount_rate('Gold'));
```

This test demonstrates how both functions work together, where the **Gold membership returns a 10% discount**, resulting in a **final price of 900**.

3.3 Stored Procedures

3.3.1 sp_update_inventory

This code defines a **stored procedure (sp_update_inventory)** that updates the **book inventory after a purchase** while ensuring proper **validation and error handling**. The procedure takes two input parameters: the **Book ID (p_BookID)** and the **quantity to be deducted (p_Quantity)**. It first checks whether the **book exists** in the Books table using a count query; if not, it raises a **custom error using SIGNAL SQLSTATE '45000'**. It then validates that the **quantity is greater than zero**, preventing invalid input. If these checks pass, the procedure retrieves the **current stock (v_stock)** and verifies whether **sufficient stock is available**. If the requested quantity exceeds the available

stock, another **error is raised to prevent negative inventory**. Otherwise, the stock is **updated by subtracting the quantity**, and a **success message is returned**.

To verify the procedure, the **stock is checked before and after execution**, confirming that the inventory is correctly reduced. Overall, this procedure ensures **data integrity, controlled updates**, and **robust validation** in the inventory management process.

3.3.2 sp_place_order

This stored procedure **sp_place_order** handles the **complete order placement process**, including **validation, pricing, order creation, and inventory update**. The procedure takes three inputs: **Customer ID, Book ID, and Quantity**, and first validates that the **quantity is greater than zero**. It then retrieves the **book's price and available stock**, ensuring that **sufficient stock is available**; otherwise, it raises **errors using SIGNAL SQLSTATE '45000'**. Next, it fetches the **customer's membership type** and validates that the **customer exists**.

The procedure then performs **pricing calculations**, where it determines the **discount rate using fn_get_discount_rate**, calculates the **original total (price × quantity)**, and computes the **final discounted amount using fn_discounted_price**. After this, it creates a new record in the **Orders table** (with status '**Pending**') and captures the **generated OrderID**. It also inserts corresponding data into the **OrderDetails table**, linking the order with the selected book and quantity.

The procedure then updates the inventory by calling **sp_update_inventory**, ensuring that the **stock is reduced safely**. Finally, it returns key outputs such as the **Order ID, original amount, discount percentage, and final amount**, allowing verification of the transaction. The process can be validated by checking the **stock before and after execution**, confirming that the inventory is correctly updated.

3.3.3 sp_process_payment

This stored procedure **sp_process_payment** handles the **payment processing for an order**, including **validation, payment recording, and order status updates**. The procedure takes three

inputs: **Order ID (p_OrderID)**, **Payment Method (p_PaymentMethod)**, and **Payment Status (p_PaymentStatus)**. It first validates whether the **order exists** by checking the **Orders table** and retrieving the **total order amount (v_total)**; if no order is found, it raises a **custom error using SIGNAL SQLSTATE '45000'**. It then validates that the **payment status is valid** (must be **'Completed', 'Pending', or 'Failed'**), ensuring only acceptable values are processed.

Once validation is complete, the procedure inserts a new record into the **Payments table**, capturing the **payment date, amount, method, status, and associated Order ID**. After recording the payment, the procedure updates the **order status based on the payment result**: if the payment is **'Completed'**, the order is marked as **'Paid'**; if **'Failed'**, the order is **'Cancelled'**; and if **'Pending'**, the order remains **'Pending'**.

Finally, the procedure can be verified by checking the **Orders and Payments tables before and after execution**, ensuring that the **payment is correctly recorded and the order status is updated accordingly**. Overall, this procedure ensures **data integrity, controlled payment processing, and accurate order lifecycle management** within the system.

3.4 Event

3.4.1 evt_cancel_unpaid_orders — Purpose & Schedule

An automated MySQL event named **evt_cancel_unpaid_orders** was created to manage unpaid orders in the Online Bookstore Management System. The event scheduler is enabled so that the event runs automatically every hour. During each run, the system checks the **Orders table** for transactions with a **Pending** status that has remained unpaid for more than **48 hours**. These outdated unpaid orders are then selected for cancellation and removal. This automation reduces manual administrative work, keeps the order records accurate, and improves the overall efficiency of the database system.

3.4.2 Step-by-Step Logic (Trigger → Restore Stock → Delete Orders)

When the event is triggered, it first identifies all pending orders that are older than 48 hours. Once these expired unpaid orders are found, the system restores the reserved book quantities by adding the ordered amounts back into the **Bookstable**, ensuring that inventory levels remain

accurate. After restoring stock, the **InventoryRestored** field in the **Orderstable** is updated to indicate that stock has been returned. The system then removes related records from the **Payments** and **OrderDetails** tables before deleting the unpaid orders from the **Orders** table. This sequence is important because it maintains referential integrity, prevents foreign key errors, and ensures that outdated unpaid transactions are removed safely and efficiently.

3.5 User Roles & Permissions

This is the most crucial component of database security. Three unique user roles are created in the Online Bookstore Management System. Permission will allow users to access the data what they are authorized for. This will ensure data security.

3.5.1 Admin Role (bookstore_admin): This role is created for the database administrator, which allows the admin to manage the entire database.

Permissions granted:

- Can access entire database like SELECT, INSERT, UPDATE, DELETE
- Execution to all SPs and functions
- Management of users and grant permission

3.5.2 Seller Role (bookstore_seller): This role is designed for staff members in a way that they can view the entire database but cannot modify any records.

Permissions granted:

- Access to read the tables only i.e. SELECT
- Can not do INSERT, UPDATE or DELETE
- Can view customer orders, book inventory, payment records, and all other data

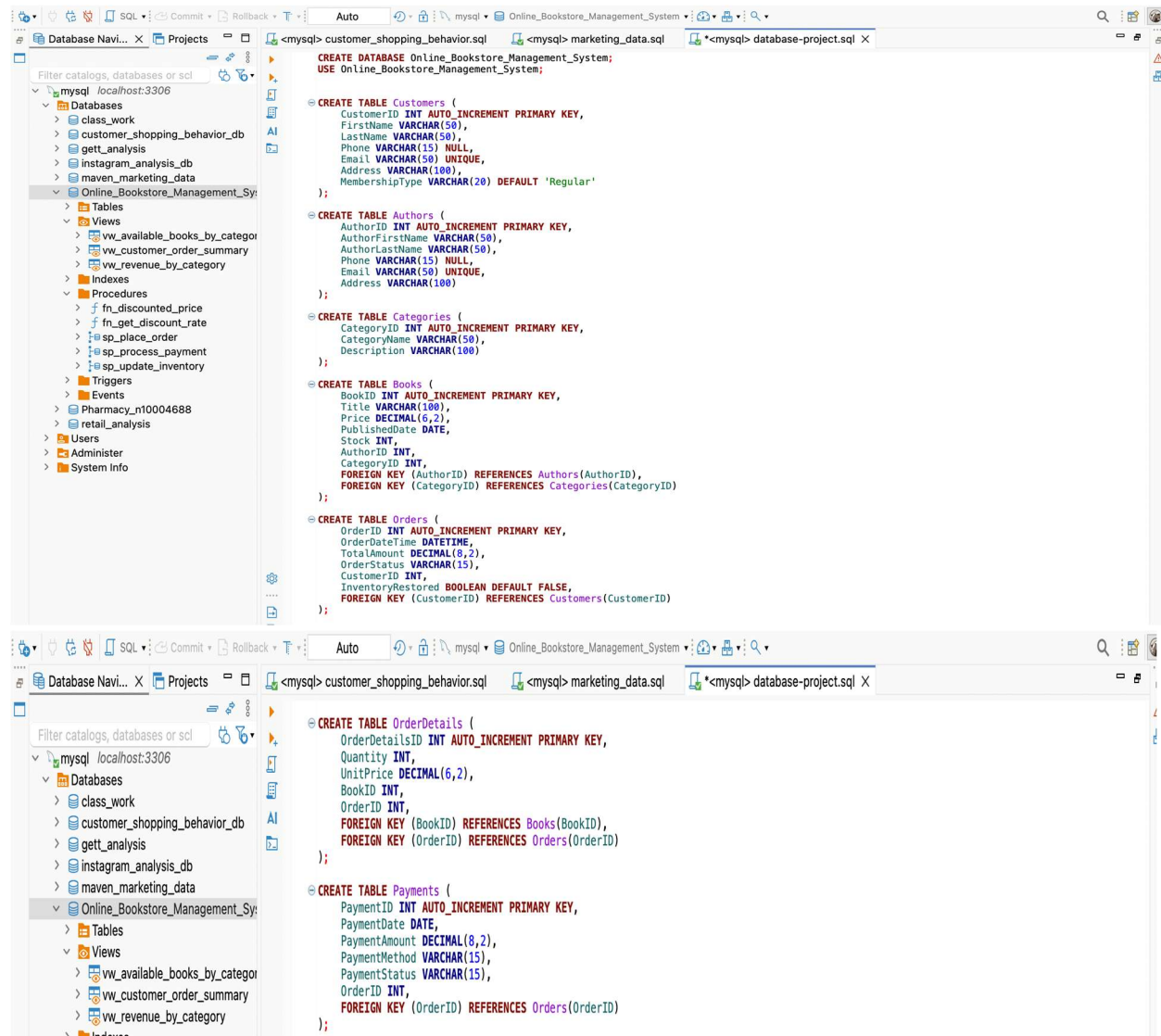
3.4.3 Customer Role (bookstore_customer): This role is one of the restricted roles in the system. It only allows customers to browse the books and their prices.

Permissions granted:

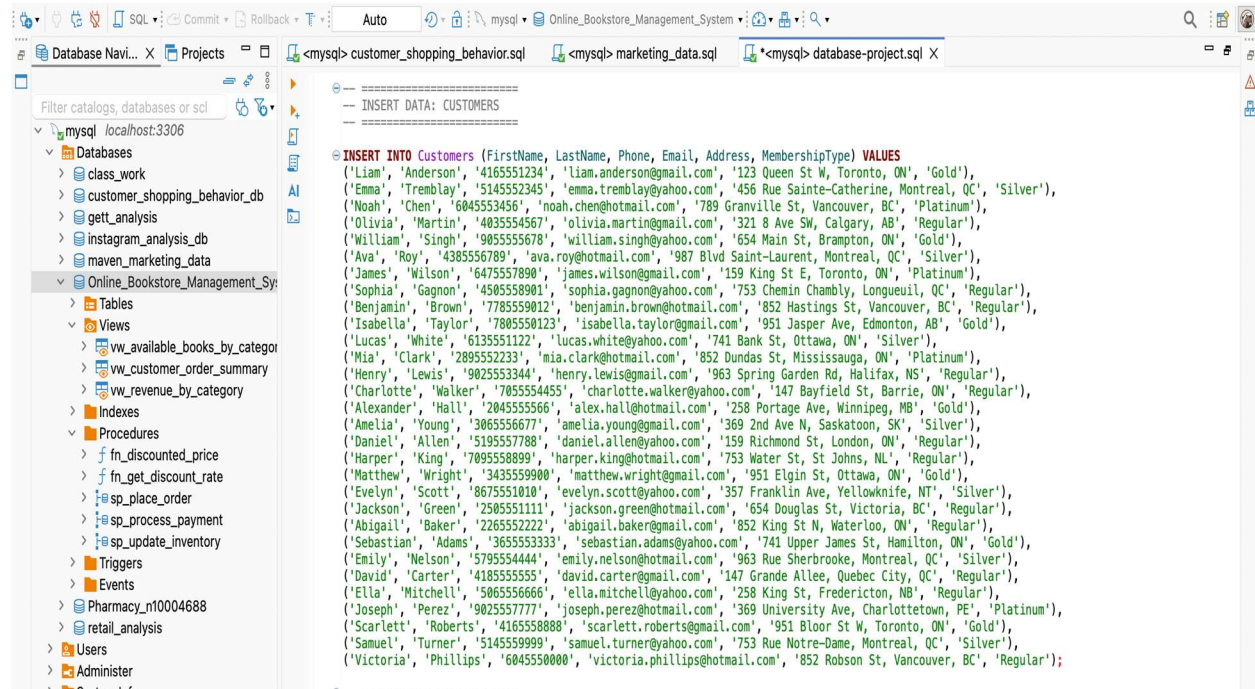
- Access to read the specific table columns only i.e. SELECT on specific columns BookID, Title, Price, Stock, CategoryID
- Can not do INSERT, UPDATE or DELETE
- Cannot access Orders, Payments, Customers, Authors, or OrderDetails tables

4. Appendix A — Screenshots

4.1 Database and Tables Created:

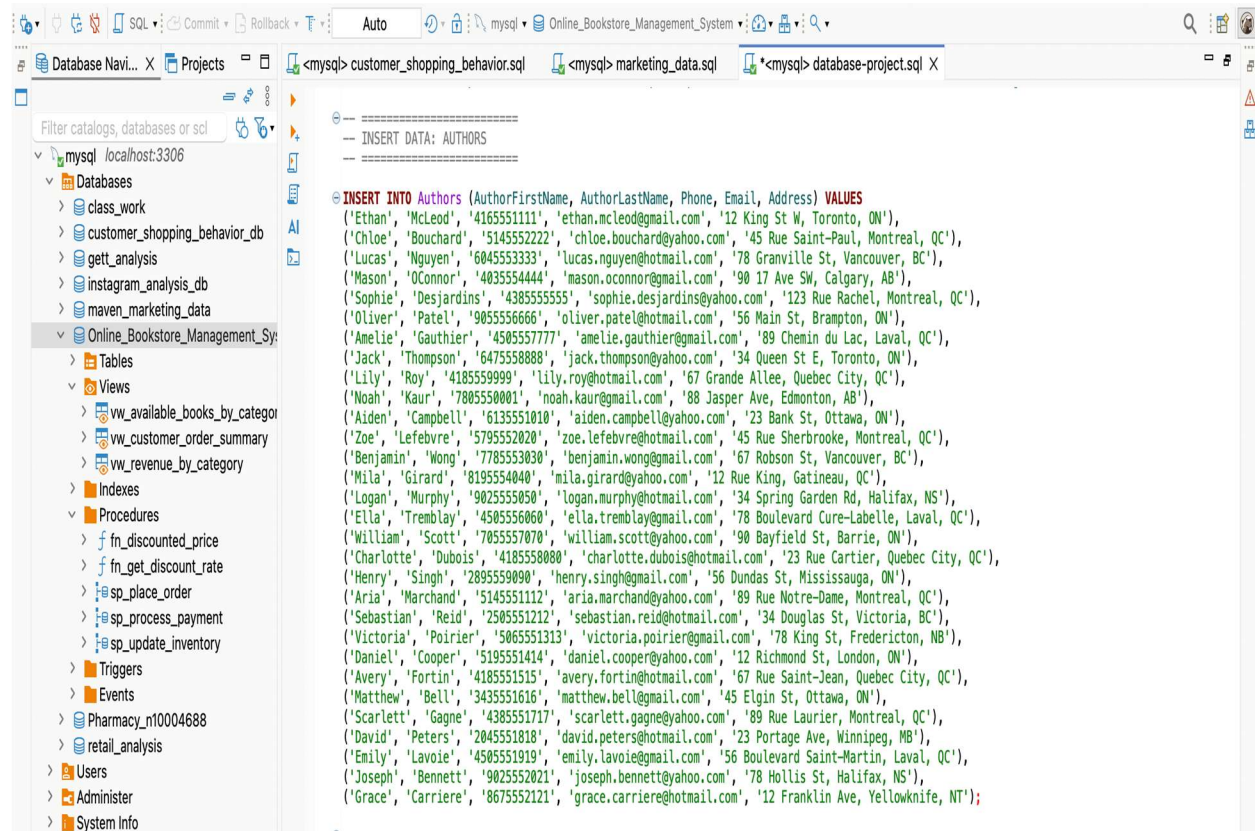


4.2 Sample Data Inserted:



The screenshot shows the Database Navigator on the left, displaying the structure of the `Online_Bookstore_Management_System` database. The right pane shows an SQL editor with the following `INSERT INTO Customers` statement:

```
INSERT INTO Customers (FirstName, LastName, Phone, Email, Address, MembershipType) VALUES
('Liam', 'Anderson', '4165551234', 'liam.anderson@gmail.com', '123 Queen St W, Toronto, ON', 'Gold'),
('Emma', 'Trenblay', '5145552345', 'emma.trenblay@yahoo.com', '456 Rue Sainte-Catherine, Montreal, QC', 'Silver'),
('Noah', 'Chen', '6045553456', 'noah.chen@hotmail.com', '789 Granville St, Vancouver, BC', 'Platinum'),
('Olivia', 'Martin', '4035554567', 'olivia.martin@gmail.com', '321 8 Ave SW, Calgary, AB', 'Regular'),
('William', 'Singh', '9055555678', 'william.singh@yahoo.com', '654 Main St, Brampton, ON', 'Gold'),
('Ava', 'Roy', '4385556789', 'ava.roy@hotmail.com', '987 Blvd Saint-Laurent, Montreal, QC', 'Silver'),
('James', 'Wilson', '6475557890', 'james.wilson@gmail.com', '159 King St E, Toronto, ON', 'Platinum'),
('Sophia', 'Gagnon', '4505558901', 'sophia.gagnon@yahoo.com', '753 Chemin Chambly, Longueuil, QC', 'Regular'),
('Benjamin', 'Brown', '7785559012', 'benjamin.brown@hotmail.com', '852 Hastings St, Vancouver, BC', 'Regular'),
('Isabella', 'Taylor', '7805550123', 'isabella.taylor@gmail.com', '951 Jasper Ave, Edmonton, AB', 'Gold'),
('Lucas', 'White', '6135551122', 'lucas.white@yahoo.com', '741 Bank St, Ottawa, ON', 'Silver'),
('Mia', 'Clark', '2895552233', 'mia.clark@hotmail.com', '852 Dundas St, Mississauga, ON', 'Platinum'),
('Henry', 'Lewis', '9025553344', 'henry.lewis@gmail.com', '963 Spring Garden Rd, Halifax, NS', 'Regular'),
('Charlotte', 'Walker', '7055554455', 'charlotte.walker@yahoo.com', '147 Bayfield St, Barrie, ON', 'Regular'),
('Alexander', 'Hall', '2045555566', 'alex.hall@hotmail.com', '258 Portage Ave, Winnipeg, MB', 'Gold'),
('Amelia', 'Young', '3065556677', 'amelia.young@gmail.com', '369 2nd Ave N, Saskatoon, SK', 'Silver'),
('Daniel', 'Allen', '5195557788', 'daniel.allen@yahoo.com', '159 Richmond St, London, ON', 'Regular'),
('Harper', 'King', '7095558899', 'harper.king@hotmail.com', '753 Water St, St Johns, NL', 'Regular'),
('Matthew', 'Wright', '3435559900', 'matthew.wright@gmail.com', '951 Elgin St, Ottawa, ON', 'Gold'),
('Evelyn', 'Scott', '8675551010', 'evelyn.scott@yahoo.com', '357 Franklin Ave, Yellowknife, NT', 'Silver'),
('Jackson', 'Green', '2505551111', 'jackson.green@hotmail.com', '654 Douglas St, Victoria, BC', 'Regular'),
('Abigail', 'Baker', '2265552222', 'abigail.baker@gmail.com', '852 King St N, Waterloo, ON', 'Regular'),
('Sebastian', 'Adams', '3655553333', 'sebastian.adams@yahoo.com', '741 Upper James St, Hamilton, ON', 'Gold'),
('Emily', 'Nelson', '5795554444', 'emily.nelson@hotmail.com', '963 Rue Sherbrooke, Montreal, QC', 'Silver'),
('David', 'Carter', '4185555555', 'david.carter@gmail.com', '147 Grande Allee, Quebec City, QC', 'Regular'),
('Ella', 'Mitchell', '5065556666', 'ella.mitchell@yahoo.com', '258 King St, Fredericton, NB', 'Regular'),
('Joseph', 'Perez', '9025557777', 'joseph.perez@hotmail.com', '369 University Ave, Charlottetown, PE', 'Platinum'),
('Scarlett', 'Roberts', '4165558888', 'scarlett.roberts@gmail.com', '951 Bloor St W, Toronto, ON', 'Gold'),
('Samuel', 'Turner', '5145559999', 'samuel.turner@yahoo.com', '753 Rue Notre-Dame, Montreal, QC', 'Silver'),
('Victoria', 'Phillips', '6045550000', 'victoria.phillips@hotmail.com', '852 Robson St, Vancouver, BC', 'Regular');
```



The screenshot shows the Database Navigator on the left, displaying the structure of the `Online_Bookstore_Management_System` database. The right pane shows an SQL editor with the following `INSERT INTO Authors` statement:

```
INSERT INTO Authors (AuthorFirstName, AuthorLastName, Phone, Email, Address) VALUES
('Ethan', 'McLeod', '4165551111', 'ethan.mcleod@gmail.com', '12 King St W, Toronto, ON'),
('Chloe', 'Bouchard', '5145552222', 'chloe.bouchard@yahoo.com', '45 Rue Saint-Paul, Montreal, QC'),
('Lucas', 'Nguyen', '6045553333', 'lucas.nguyen@hotmail.com', '78 Granville St, Vancouver, BC'),
('Mason', 'O'Connor', '4035554444', 'mason.oconnor@gmail.com', '90 17 Ave SW, Calgary, AB'),
('Sophie', 'Desjardins', '4385555555', 'sophie.desjardins@yahoo.com', '123 Rue Rachel, Montreal, QC'),
('Oliver', 'Patel', '9055556666', 'oliver.patel@hotmail.com', '56 Main St, Brampton, ON'),
('Amelie', 'Gauthier', '4505557777', 'amelie.gauthier@gmail.com', '89 Chemin du Lac, Laval, QC'),
('Jack', 'Thompson', '6475558888', 'jack.thompson@yahoo.com', '34 Queen St E, Toronto, ON'),
('Lily', 'Roy', '4185559999', 'lily.roy@hotmail.com', '67 Grande Allee, Quebec City, QC'),
('Noah', 'Kaur', '7805550001', 'noah.kaur@gmail.com', '88 Jasper Ave, Edmonton, AB'),
('Aiden', 'Campbell', '6135551010', 'aiden.campbell@yahoo.com', '23 Bank St, Ottawa, ON'),
('Zoe', 'Lefebvre', '5795552020', 'zoe.lefebvre@hotmail.com', '45 Rue Sherbrooke, Montreal, QC'),
('Benjamin', 'Wong', '7785553030', 'benjamin.wong@gmail.com', '67 Robson St, Vancouver, BC'),
('Mila', 'Girard', '8195554040', 'mila.girard@yahoo.com', '12 Rue King, Gatineau, QC'),
('Logan', 'Murphy', '9025555050', 'logan.murphy@hotmail.com', '34 Spring Garden Rd, Halifax, NS'),
('Ella', 'Trenblay', '4505556060', 'ella.trenblay@gmail.com', '78 Boulevard Cure-Labelle, Laval, QC'),
('William', 'Scott', '7055557070', 'william.scott@yahoo.com', '90 Bayfield St, Barrie, ON'),
('Charlotte', 'Dubois', '4185558080', 'charlotte.dubois@hotmail.com', '23 Rue Cartier, Quebec City, QC'),
('Henry', 'Singh', '2895559090', 'henry.singh@gmail.com', '56 Dundas St, Mississauga, ON'),
('Aria', 'Marchand', '5145551112', 'aria.marchand@yahoo.com', '89 Rue Notre-Dame, Montreal, QC'),
('Sebastian', 'Reid', '2505551212', 'sebastian.reid@hotmail.com', '34 Douglas St, Victoria, BC'),
('Victoria', 'Poirier', '5065551313', 'victoria.poirier@gmail.com', '78 King St, Fredericton, NB'),
('Daniel', 'Cooper', '5195551414', 'daniel.cooper@yahoo.com', '12 Richmond St, London, ON'),
('Avery', 'Fortin', '4185551515', 'avery.fortin@hotmail.com', '67 Rue Saint-Jean, Quebec City, QC'),
('Matthew', 'Bell', '3435551616', 'matthew.bell@gmail.com', '45 Elgin St, Ottawa, ON'),
('Scarlett', 'Gagne', '4385551717', 'scarlett.gagne@yahoo.com', '89 Rue Laurier, Montreal, QC'),
('David', 'Peters', '2045551818', 'david.peters@hotmail.com', '23 Portage Ave, Winnipeg, MB'),
('Emily', 'Lavoie', '4505551919', 'emily.lavoie@gmail.com', '56 Boulevard Saint-Martin, Laval, QC'),
('Joseph', 'Bennett', '9025552021', 'joseph.bennett@yahoo.com', '78 Hollis St, Halifax, NS'),
('Grace', 'Carriere', '8675552121', 'grace.carriere@hotmail.com', '12 Franklin Ave, Yellowknife, NT');
```

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - gett_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer

```

=====
-- INSERT DATA: CATEGORIES
=====

INSERT INTO Categories (CategoryName, Description) VALUES
('Fiction', 'Imaginary stories including novels and short stories'),
('Non-Fiction', 'Informative books based on real facts and events'),
('Science Fiction', 'Futuristic concepts, space, and advanced technology'),
('Fantasy', 'Magic, mythical creatures, and imaginary worlds'),
('Mystery', 'Suspenseful stories involving solving crimes or puzzles'),
('Thriller', 'Fast-paced stories with tension and excitement'),
('Romance', 'Love stories and emotional relationships'),
('Historical Fiction', 'Stories set in historical time periods'),
('Biography', 'Life stories of real people'),
('Autobiography', 'Self-written life stories'),
('Self-Help', 'Books focused on personal development and growth'),
('Business', 'Entrepreneurship, management, and finance topics'),
('Technology', 'Books about computers, IT, and modern tech'),
('Programming', 'Coding languages, software development, and algorithms'),
('Health & Fitness', 'Wellness, exercise, and healthy living'),
('Cooking', 'Recipes, culinary skills, and food culture'),
('Travel', 'Guides and experiences from around the world'),
('Children', 'Books for young readers and kids'),
('Young Adult', 'Books targeted toward teenagers'),
('Education', 'Academic and learning materials'),
('Religion & Spirituality', 'Beliefs, faith, and spiritual practices'),
('Science', 'Physics, chemistry, biology, and research topics'),
('History', 'Events and civilizations from the past'),
('Art & Photography', 'Creative arts, design, and visual storytelling'),
('Comics & Graphic Novels', 'Illustrated stories and comic books'),
('Poetry', 'Collections of poems and literary expressions'),
('Horror', 'Scary and supernatural stories'),
('Drama', 'Serious narratives with emotional themes'),
('Sports', 'Books about sports, athletes, and training'),
('Politics', 'Government, policies, and political analysis');
=====

```

Database Navigator: Filter catalogs, databases or schema

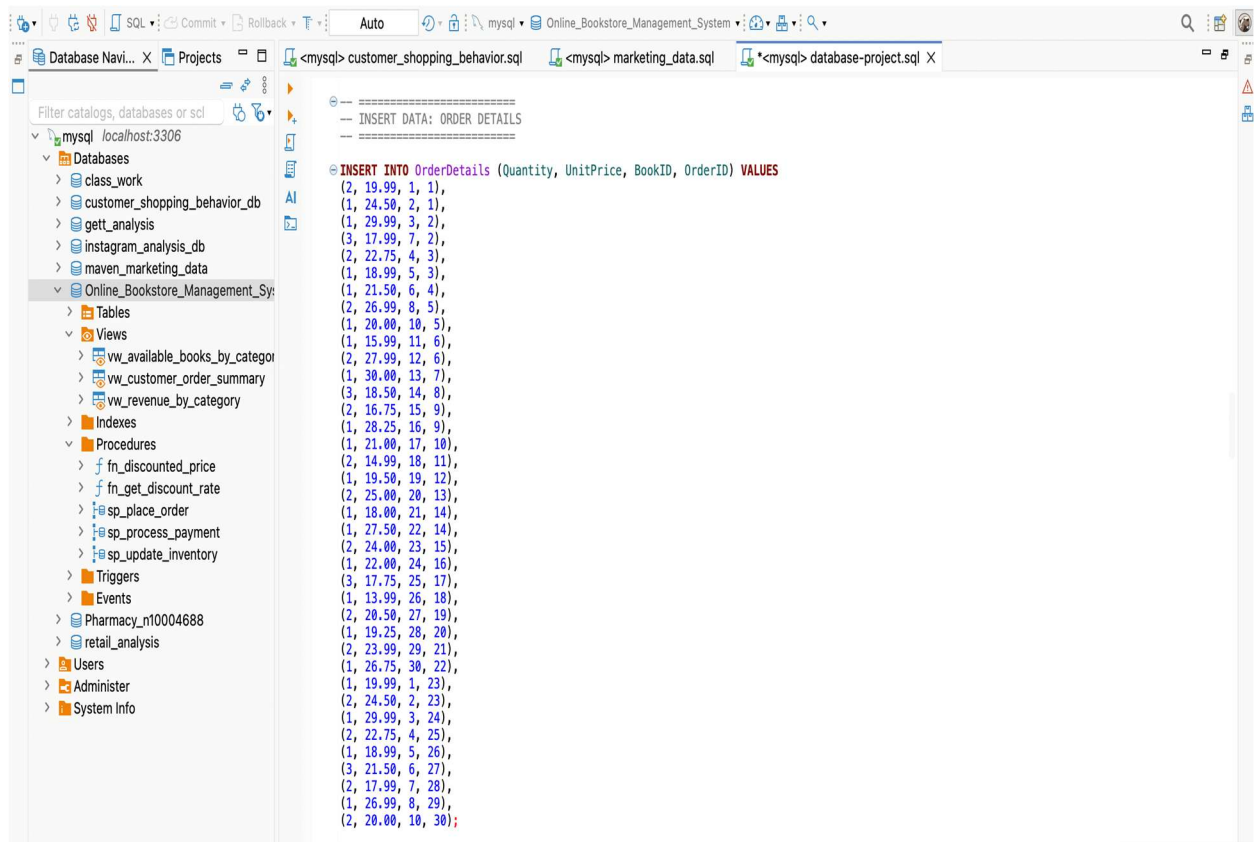
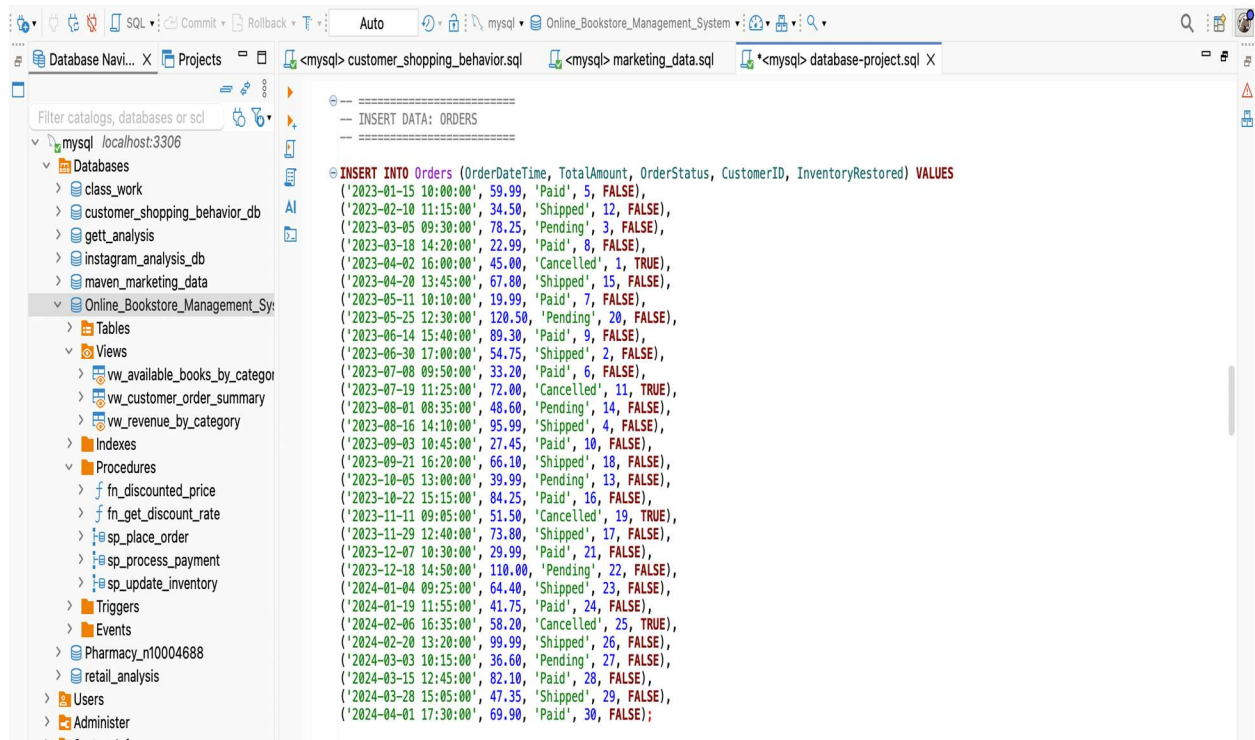
- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - gett_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

=====
-- INSERT DATA: BOOKS
=====

INSERT INTO Books (Title, Price, PublishedDate, Stock, AuthorID, CategoryID) VALUES
('The Silent Forest', 19.99, '2020-05-12', 50, 5, 12),
('Beyond the Horizon', 24.50, '2019-08-23', 40, 2, 7),
('Galactic Wars', 29.99, '2021-03-15', 35, 14, 3),
('Dragon Realm', 22.75, '2018-11-10', 60, 8, 4),
('The Hidden Clue', 18.99, '2020-01-05', 45, 11, 5),
('Final Escape', 21.50, '2022-06-20', 30, 3, 6),
('Love in Paris', 17.99, '2017-02-14', 55, 7, 7),
('War of Kings', 26.99, '2016-09-09', 25, 15, 8),
('Life of a Genius', 23.50, '2021-12-01', 20, 9, 9),
('My Journey', 20.00, '2019-04-18', 28, 1, 10),
('Better You', 15.99, '2022-01-10', 70, 6, 11),
('Startup Secrets', 27.99, '2021-07-07', 33, 12, 12),
('Future Tech', 30.00, '2023-03-03', 22, 4, 13),
('Learn SQL Fast', 18.50, '2022-10-10', 80, 14, 14),
('Healthy Living Guide', 16.75, '2020-06-06', 65, 10, 15),
('MasterChef Recipes', 28.25, '2018-12-12', 40, 16, 16),
('World Explorer', 21.00, '2019-05-25', 35, 18, 17),
('Kids Fun Stories', 14.99, '2021-11-11', 90, 20, 18),
('Teen Dreams', 19.50, '2020-08-08', 50, 19, 19),
('Math Basics', 25.00, '2017-09-01', 60, 13, 20),
('Spiritual Path', 18.00, '2016-03-03', 45, 21, 21),
('Physics Simplified', 27.50, '2022-04-04', 30, 22, 22),
('Ancient Civilizations', 24.00, '2018-07-07', 38, 17, 23),
('Creative Arts', 22.00, '2021-09-09', 27, 24, 24),
('Superhero Tales', 17.75, '2020-10-10', 75, 25, 25),
('Poems of Life', 13.99, '2019-01-01', 55, 23, 26),
('Night Terrors', 20.50, '2023-02-02', 20, 27, 27),
('Family Drama', 19.25, '2017-06-06', 42, 26, 28),
('Champions Mindset', 23.99, '2021-03-21', 33, 29, 29),
('Global Politics', 26.75, '2022-05-05', 29, 30, 30);
=====

```

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - gett_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

=====
-- INSERT DATA: PAYMENTS
=====

INSERT INTO Payments (PaymentDate, PaymentAmount, PaymentMethod, PaymentStatus, OrderID) VALUES
('2023-01-15', 59.99, 'Credit Card', 'Completed', 1),
('2023-02-10', 34.50, 'PayPal', 'Completed', 2),
('2023-03-06', 78.25, 'Debit Card', 'Pending', 3),
('2023-03-18', 22.99, 'Credit Card', 'Completed', 4),
('2023-04-02', 45.00, 'Cash', 'Failed', 5),
('2023-04-20', 67.00, 'Debit Card', 'Completed', 6),
('2023-05-11', 19.99, 'Credit Card', 'Completed', 7),
('2023-05-25', 120.50, 'PayPal', 'Pending', 8),
('2023-06-14', 89.30, 'Credit Card', 'Completed', 9),
('2023-06-30', 54.75, 'Debit Card', 'Completed', 10),
('2023-07-08', 33.20, 'Credit Card', 'Completed', 11),
('2023-07-19', 72.00, 'Cash', 'Failed', 12),
('2023-08-01', 48.60, 'PayPal', 'Pending', 13),
('2023-08-16', 95.99, 'Debit Card', 'Completed', 14),
('2023-09-03', 27.45, 'Credit Card', 'Completed', 15),
('2023-09-21', 66.10, 'PayPal', 'Completed', 16),
('2023-10-05', 39.99, 'Debit Card', 'Pending', 17),
('2023-10-22', 84.25, 'Credit Card', 'Completed', 18),
('2023-11-11', 51.50, 'Cash', 'Failed', 19),
('2023-11-29', 73.00, 'Debit Card', 'Completed', 20),
('2023-12-07', 29.99, 'Credit Card', 'Completed', 21),
('2023-12-18', 110.00, 'PayPal', 'Pending', 22),
('2024-01-04', 64.40, 'Debit Card', 'Completed', 23),
('2024-01-19', 41.75, 'Credit Card', 'Completed', 24),
('2024-02-06', 58.20, 'Cash', 'Failed', 25),
('2024-02-20', 99.99, 'Debit Card', 'Completed', 26),
('2024-03-03', 36.00, 'PayPal', 'Pending', 27),
('2024-03-15', 82.10, 'Credit Card', 'Completed', 28),
('2024-03-28', 47.35, 'Debit Card', 'Completed', 29),
('2024-04-01', 69.90, 'Credit Card', 'Completed', 30);
  
```

4.3 Views Executed:

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - gett_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

FROM OrderDetails od
JOIN Books b
  ON od.BookID = b.BookID
JOIN Categories c
  ON b.CategoryID = c.CategoryID
JOIN Orders o
  ON od.OrderID = o.OrderID
WHERE o.OrderStatus IN ('Paid', 'Shipped')
GROUP BY c.CategoryID, c.CategoryName;

-- Execute All views
=====
SELECT * FROM vw_available_books_by_category;
SELECT * FROM vw_customer_order_summary;
SELECT * FROM vw_revenue_by_category;

-- User-Defined Functions (UDFs)
=====
-- 4b.1 fn_get_discount_rate
  
```

vw_available_books_by_category 1 X

SELECT * FROM vw_available_books_by_category

Grid	123 BookID	A2 Title	A2 CategoryName	A2 AuthorName	123 Price	123 Stock
1	1	The Silent Forest	Business	Sophie Desjardins	19.99	48
2	2	Beyond the Horizon	Romance	Chloe Bouchard	24.5	40
3	3	Galactic Wars	Science Fiction	Mila Girard	29.99	33
4	4	Dragon Realm	Fantasy	Jack Thompson	22.75	60
5	5	The Hidden Clue	Mystery	Aiden Campbell	18.99	45
6	6	Final Escape	Thriller	Lucas Nguyen	21.5	30
7	7	Love in Paris	Romance	Amelie Gauthier	17.99	55
8	8	War of Kings	Historical Fiction	Logan Murphy	26.99	25
9	9	Life of a Genius	Biography	Lily Roy	23.5	20
10	10	My Journey	Autobiography	Ethan McLeod	20	28
11	11	Better You	Self-Help	Oliver Patel	15.99	70
12	12	Startup Secrets	Business	Zoe Lefebvre	27.99	33
13	13	Future Tech	Technology	Mason OConnor	30	22
14	14	Learn SQL Fast	Programming	Mila Girard	18.5	80
15	15	Healthy Living Guide	Health & Fitness	Noah Kaur	16.75	65
16	16	MasterChef Recipes	Cooking	Ella Tremblay	28.25	40
17	17	World Explorer	Travel	Charlotte Dubois	21	35
18	18	Kids Fun Stories	Children	Aria Marchand	14.99	90

Refresh Save Cancel Export data 200 30

EDT en Writable Smart Insert 391:1:17711

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

SQL Editor: customer_shopping_behavior.sql

```

FROM OrderDetails od
JOIN Books b
  ON od.BookID = b.BookID
JOIN Categories c
  ON b.CategoryID = c.CategoryID
JOIN Orders o
  ON od.OrderID = o.OrderID
WHERE o.OrderStatus IN ('Paid', 'Shipped')
GROUP BY c.CategoryID, c.CategoryName;
  
```

Execute All views

SELECT * FROM vw_available_books_by_category;
 SELECT * FROM vw_customer_order_summary;
 SELECT * FROM vw_revenue_by_category;

User-Defined Functions (UDFs)

4b.1 fn_get_discount_rate

categories(+) 1 X

SELECT * FROM vw_revenue_by_category

CategoryID	CategoryName	TotalBooksSold	TotalRevenue
12	Business	7	155.93
7	Romance	8	163.45
3	Science Fiction	2	59.98
6	Thriller	1	21.5
11	Self-Help	1	15.99
13	Technology	1	30
15	Health & Fitness	2	33.5
16	Cooking	1	28.25
17	Travel	1	21
18	Children	2	29.98
21	Religion & Spirituality	1	18
22	Science	1	27.5
23	History	2	48
24	Art & Photography	1	22
26	Poetry	1	13.99
28	Drama	1	19.25
29	Sports	2	47.98

Refresh Save Cancel Export data 200 20

20 row(s) fetched - 0.016s (0.001s fetch), on 2026-04-24 at 20:02:36

EDT en Writable Smart Insert 393 : 1 : 17798

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

SQL Editor: customer_shopping_behavior.sql

```

FROM OrderDetails od
JOIN Books b
  ON od.BookID = b.BookID
JOIN Categories c
  ON b.CategoryID = c.CategoryID
JOIN Orders o
  ON od.OrderID = o.OrderID
WHERE o.OrderStatus IN ('Paid', 'Shipped')
GROUP BY c.CategoryID, c.CategoryName;
  
```

Execute All views

SELECT * FROM vw_available_books_by_category;
 SELECT * FROM vw_customer_order_summary;
 SELECT * FROM vw_revenue_by_category;

User-Defined Functions (UDFs)

4b.1 fn_get_discount_rate

vw_customer_order_summary 1 X

SELECT * FROM vw_customer_order_summary

OrderID	OrderDate	CustomerName	TotalAmount	OrderStatus	InventoryRestored
5	2023-04-02 16:00:00	Liam Anderson	45	Cancelled	1
31	2026-04-24 19:35:38	Liam Anderson	35.98	Paid	0
10	2023-06-30 17:00:00	Emma Tremblay	54.75	Shipped	0
3	2023-03-05 09:30:00	Noah Chen	78.25	Pending	0
14	2023-08-16 14:10:00	Olivia Martin	95.99	Shipped	0
1	2023-01-15 10:00:00	William Singh	59.99	Paid	0
11	2023-07-08 09:50:00	Ava Roy	33.2	Paid	0
7	2023-05-11 10:10:00	James Wilson	19.99	Paid	0
4	2023-03-18 14:20:00	Sophia Gagnon	22.99	Paid	0
9	2023-06-14 15:40:00	Benjamin Brown	89.3	Paid	0
15	2023-09-03 10:45:00	Isabella Taylor	27.45	Paid	0
12	2023-07-19 11:25:00	Lucas White	72	Cancelled	1
2	2023-02-10 11:15:00	Mia Clark	34.5	Shipped	0
17	2023-10-05 13:00:00	Henry Lewis	39.99	Pending	0
13	2023-08-01 08:35:00	Charlotte Walker	48.6	Pending	0
6	2023-04-20 13:45:00	Alexander Hall	67.8	Shipped	0
18	2023-10-22 15:15:00	Amelia Young	84.25	Paid	0

Refresh Save Cancel Export data 200 31

31 row(s) fetched - 0.031s (0.003s fetch), on 2026-04-24 at 20:02:13

EDT en Writable Smart Insert 392 : 1 : 17757

4.4 UDFs Tested

fn_get_discount_rate

```
-- =====
-- User-Defined Functions (UDFs)
-- =====

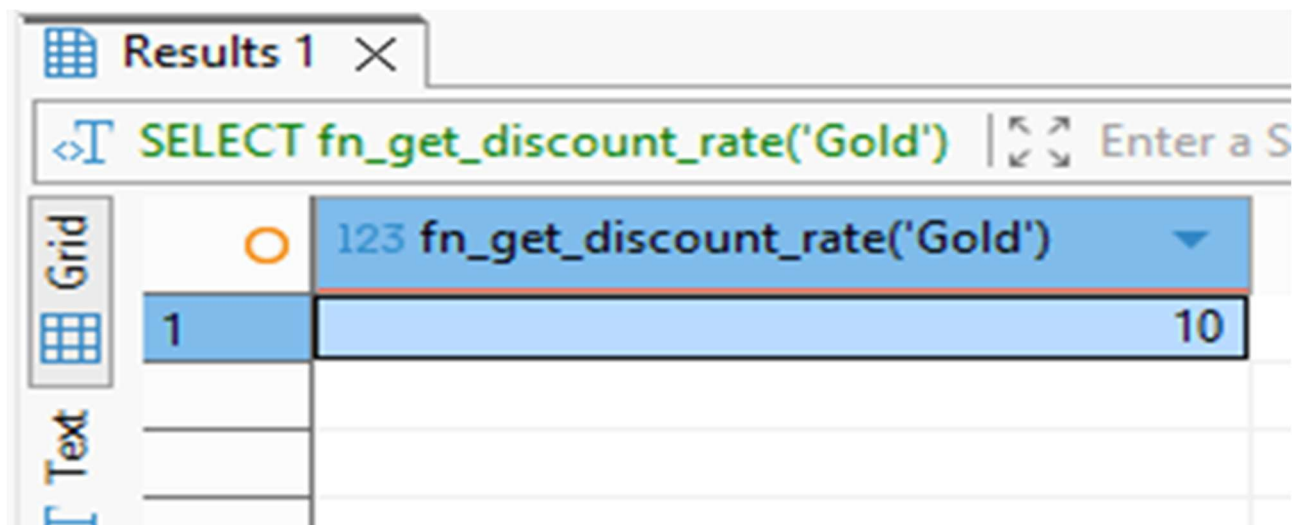
-- 4b.1 fn_get_discount_rate

-- Create a user-defined function to return discount rate based on membership type
CREATE FUNCTION fn_get_discount_rate(p_membership VARCHAR(20))
RETURNS DECIMAL(5,2)
DETERMINISTIC -- Indicates the function always returns the same output for the same input
BEGIN
    -- Declare a variable to store the calculated discount percentage
    DECLARE v_discount DECIMAL(5,2);

    -- Assign discount based on membership type using CASE logic
    SET v_discount =
        CASE
            -- Convert input to uppercase to make comparison case-insensitive
            WHEN UPPER(p_membership) = 'SILVER' THEN 5.00 -- 5% discount
            WHEN UPPER(p_membership) = 'GOLD' THEN 10.00 -- 10% discount
            WHEN UPPER(p_membership) = 'PLATINUM' THEN 15.00 -- 15% discount
            ELSE 0.00 -- No discount for other membership types
        END;

    -- Return the calculated discount value
    RETURN v_discount;
END;
```

```
-- Test the function by passing a membership type
SELECT fn_get_discount_rate('Gold') -- output: 10.00 ;
```



The screenshot shows a database client window titled "Results 1". The SQL query entered is `SELECT fn_get_discount_rate('Gold')`. The results are displayed in a grid view with one column and one row. The column header is `fn_get_discount_rate('Gold')` and the value in the row is `10`.

	fn_get_discount_rate('Gold')
1	10

fn_discounted_price

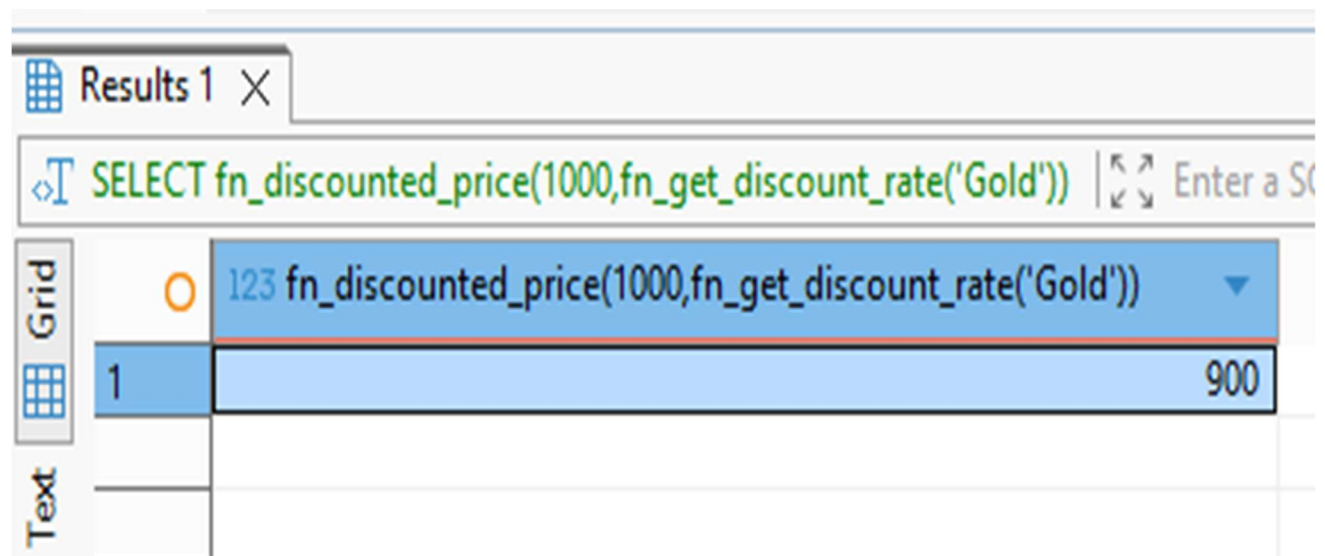
```
-- 4b.2 fn_discounted_price

-- Create a user-defined function to calculate the final price after applying a discount
CREATE FUNCTION fn_discounted_price(
    p_original_price DECIMAL(10,2), -- Input parameter: original price of the product
    p_discount_rate DECIMAL(5,2)    -- Input parameter: discount percentage (e.g., 10 for 10%)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC -- Ensures the function always returns the same output for the same inputs
BEGIN
    -- Declare a variable to store the computed final price after discount
    DECLARE final_price DECIMAL(10,2);

    -- Calculate the discounted price using the formula:
    -- Final Price = Original Price - (Original Price x Discount Rate / 100)
    SET final_price = p_original_price - (p_original_price * p_discount_rate / 100);

    -- Return the calculated final price
    RETURN final_price;
END;

-- Test the function
SELECT fn_discounted_price(1000,fn_get_discount_rate('Gold')) -- output: 900;
```



Results 1	
SELECT fn_discounted_price(1000,fn_get_discount_rate('Gold'))	
1	900

4.5 Stored Procedures Executed: sp_update_inventory

The screenshot shows the Database Navigator on the left with the 'Online_Bookstore_Management_System' database selected. The central pane displays the SQL code for the `sp_update_inventory` stored procedure. The procedure includes steps for retrieving the current stock value, checking if enough stock is available, and updating the inventory by reducing the stock. The results pane at the bottom shows the state of the `books` table before the update.

```
SET MESSAGE_TEXT = 'Quantity must be greater than zero.';

ELSE
    -- Step 4: Retrieve current stock value
    SELECT Stock INTO v_stock
    FROM Books
    WHERE BookID = p_BookID;

    -- Step 5: Check if enough stock is available
    IF v_stock < p_Quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Not enough stock available.';
    ELSE
        -- Step 6: Update inventory by reducing stock
        UPDATE Books
        SET Stock = Stock - p_Quantity
        WHERE BookID = p_BookID;

        -- Optional: return success message
        SELECT 'Inventory updated successfully' AS message;
    END IF;
END IF;

-- Check stock BEFORE
SELECT BookID, Stock
FROM Books
WHERE BookID = 3;

-- Run procedure
CALL sp_update_inventory(3, 2);

-- Check stock AFTER
SELECT BookID, Stock
FROM Books
WHERE BookID = 3;
```

BookID	Stock
1	3
2	35

The screenshot shows the Database Navigator on the left with the 'Online_Bookstore_Management_System' database selected. The central pane displays the SQL code for the `sp_update_inventory` stored procedure. The results pane at the bottom shows the state of the `books` table after the update, displaying a success message.

```
SET MESSAGE_TEXT = 'Quantity must be greater than zero.';

ELSE
    -- Step 4: Retrieve current stock value
    SELECT Stock INTO v_stock
    FROM Books
    WHERE BookID = p_BookID;

    -- Step 5: Check if enough stock is available
    IF v_stock < p_Quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Not enough stock available.';
    ELSE
        -- Step 6: Update inventory by reducing stock
        UPDATE Books
        SET Stock = Stock - p_Quantity
        WHERE BookID = p_BookID;

        -- Optional: return success message
        SELECT 'Inventory updated successfully' AS message;
    END IF;
END IF;

-- Check stock BEFORE
SELECT BookID, Stock
FROM Books
WHERE BookID = 3;

-- Run procedure
CALL sp_update_inventory(3, 2);

-- Check stock AFTER
SELECT BookID, Stock
FROM Books
WHERE BookID = 3;
```

message
Inventory updated successfully

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

-- Step 4: Retrieve current stock value
SELECT Stock INTO v_stock
FROM Books
WHERE BookID = p_BookID;

-- Step 5: Check if enough stock is available
IF v_stock < p_Quantity THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Not enough stock available.';
ELSE
  -- Step 6: Update inventory by reducing stock
  UPDATE Books
  SET Stock = Stock - p_Quantity
  WHERE BookID = p_BookID;

  -- Optional: return success message
  SELECT 'Inventory updated successfully' AS message;
END IF;
END IF;

-- Check stock BEFORE
SELECT BookID, Stock
FROM Books
WHERE BookID = 3;

-- Run procedure
CALL sp_update_inventory(3, 2);

-- Check stock AFTER
SELECT BookID, Stock
FROM Books
WHERE BookID = 3;
  
```

books 1 X

SELECT BookID, Stock FROM Books WHERE

Grid	123 BookID	123 Stock
1	3	33

Refresh Save Cancel Export data 200 1 row(s) fetched - 0.001s, on 2026-04-24 at 19:32:59

EDT en Writable Smart Insert 517 : 1 : 21729

sp_place_order

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

-- Step 7: Update inventory
CALL sp_update_inventory(p_BookID, p_Quantity);

-- Step 8: Return output
SELECT
  v_order_id AS OrderID,
  v_original_total AS OriginalAmount,
  v_discount_rate AS DiscountPercent,
  v_final_total AS FinalAmount;
END;

-- Testing
-- Step 1: Check stock before
SELECT BookID, Stock FROM Books WHERE BookID = 1;

-- Step 2: Place order "Customer 1 is ordering 2 copies of Book 1"
CALL sp_place_order(1, 1, 2);

-- Step 3: Check stock after
SELECT BookID, Stock FROM Books WHERE BookID = 1;

-- 4c.3 sp_process_payment -----
-- Create stored procedure to process a payment for an order
CREATE PROCEDURE sp_process_payment(
  IN p_OrderID INT, -- Input: Order ID for which payment is being made
  IN p_PaymentMethod VARCHAR(15), -- Input: Payment method (e.g., Credit Card, PayPal)
  IN p_PaymentStatus VARCHAR(15) -- Input: Payment status (Completed, Pending, Failed)
)
BEGIN
  -- Declare variables to store intermediate values
  
```

books 1 X

SELECT BookID, Stock FROM Books WHERE

Grid	123 BookID	123 Stock
1	1	50

Refresh Save Cancel Export data 200 1 row(s) fetched - 0.003s, on 2026-04-24 at 19:35:25

EDT en Writable Smart Insert 651 : 1 : 25077

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - gett_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

-- Step 7: Update inventory
CALL sp_update_inventory(p_BookID, p_Quantity);

-- Step 8: Return output
SELECT
  v_order_id AS OrderID,
  v_original_total AS OriginalAmount,
  v_discount_rate AS DiscountPercent,
  v_final_total AS FinalAmount;
END;

-- Testing
-- Step 1: Check stock before
SELECT BookID, Stock FROM Books WHERE BookID = 1;

-- Step 2: Place order "Customer 1 is ordering 2 copies of Book 1"
CALL sp_place_order(1, 1, 2);

-- Step 3: Check stock after
SELECT BookID, Stock FROM Books WHERE BookID = 1;

-- 4c.3 sp_process_payment -----
-- Create stored procedure to process a payment for an order
CREATE PROCEDURE sp_process_payment(
  IN p_OrderID INT, -- Input: Order ID for which payment is being made
  IN p_PaymentMethod VARCHAR(15), -- Input: Payment method (e.g., Credit Card, PayPal)
  IN p_PaymentStatus VARCHAR(15) -- Input: Payment status (Completed, Pending, Failed)
)
BEGIN
  -- Declare variables to store intermediate values

```

Results 1 | Results 1 (2) | Statistics 1

CALL sp_place_order(1, 1, 2) | Data filter is not supported

	OrderID	OriginalAmount	DiscountPercent	FinalAmount
1	31	39.98	10	35.98

Refresh | Save | Cancel | Export data | 200 | 1 | 1 row(s) fetched

EDT | en | Writable | Smart Insert | 655:1:25196

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - gett_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

```

-- Step 7: Update inventory
CALL sp_update_inventory(p_BookID, p_Quantity);

-- Step 8: Return output
SELECT
  v_order_id AS OrderID,
  v_original_total AS OriginalAmount,
  v_discount_rate AS DiscountPercent,
  v_final_total AS FinalAmount;
END;

-- Testing
-- Step 1: Check stock before
SELECT BookID, Stock FROM Books WHERE BookID = 1;

-- Step 2: Place order "Customer 1 is ordering 2 copies of Book 1"
CALL sp_place_order(1, 1, 2);

-- Step 3: Check stock after
SELECT BookID, Stock FROM Books WHERE BookID = 1;

-- 4c.3 sp_process_payment -----
-- Create stored procedure to process a payment for an order
CREATE PROCEDURE sp_process_payment(
  IN p_OrderID INT, -- Input: Order ID for which payment is being made
  IN p_PaymentMethod VARCHAR(15), -- Input: Payment method (e.g., Credit Card, PayPal)
  IN p_PaymentStatus VARCHAR(15) -- Input: Payment status (Completed, Pending, Failed)
)
BEGIN
  -- Declare variables to store intermediate values

```

books 1 |

SELECT BookID, Stock FROM Books WHERE BookID = 1

	BookID	Stock
1	1	48

Refresh | Save | Cancel | Export data | 200 | 1 | 1 row(s) fetched - 0.001s, on 2026-04-24 at 19:35:50

EDT | en | Writable | Smart Insert | 658:1:25256

sp_process_payment

The screenshot shows the Database Navigator interface with the `sp_process_payment` stored procedure selected. The procedure code is displayed in the main editor, and the results of the procedure are shown in the `orders 1` tab.

```
-- If payment failed - cancel the order
UPDATE Orders
SET OrderStatus = 'Cancelled'
WHERE OrderID = p_OrderID;

ELSEIF p_PaymentStatus = 'Pending' THEN
-- If payment still pending - keep order as Pending
UPDATE Orders
SET OrderStatus = 'Pending'
WHERE OrderID = p_OrderID;
END IF;

END;
```

Testing
Check an existing order

```
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
ORDER BY OrderID DESC;
```

Run the procedure
CALL sp_process_payment(31, 'Credit Card', 'Completed');

orders 1

OrderID	TotalAmount	OrderStatus
31	35.98	Pending
30	69.9	Paid
29	47.35	Shipped
28	82.1	Paid
27	36.6	Pending
26	99.99	Shipped
25	58.2	Cancelled
24	41.75	Paid
23	64.4	Shipped
22	110	Pending
21	29.99	Paid
20	73.8	Shipped
19	51.5	Cancelled
18	84.25	Paid
17	39.99	Pending
16	66.1	Shipped
15	27.45	Paid

The screenshot shows the Database Navigator interface with the `sp_process_payment` stored procedure selected. The procedure code is displayed in the main editor, and the statistics of the procedure are shown in the `Statistics 1` tab.

```
END;
```

Testing
Check an existing order

```
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
ORDER BY OrderID DESC;
```

Run the procedure
CALL sp_process_payment(31, 'Credit Card', 'Completed');

Check if payment was inserted

```
SELECT *
FROM Payments
ORDER BY PaymentID DESC
LIMIT 1;
```

Check if order status changed

```
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
WHERE OrderID = 31;
```

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.012s
Start time	Fri Apr 24 19:37:47 EDT 2026
Finish time	Fri Apr 24 19:37:47 EDT 2026
Query	CALL sp_process_payment(31, 'Credit Card', 'Completed')

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

SQL Editor: <mysql> customer_shopping_behavior.sql

```

END;

-- Testing
-- Check an existing order
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
ORDER BY OrderID DESC;

-- Run the procedure
CALL sp_process_payment(31, 'Credit Card', 'Completed');

-- Check if payment was inserted
SELECT *
FROM Payments
ORDER BY PaymentID DESC
LIMIT 1;

-- Check if order status changed
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
WHERE OrderID = 31;
  
```

payments 1 X

SELECT * FROM Payments ORDER BY PaymentID DESC

	123 PaymentID	PaymentDate	123 PaymentAmount	AZ PaymentMethod	AZ PaymentStatus	123 OrderID
1	31	2026-04-24	35.98	Credit Card	Completed	31

Refresh Save Cancel Export data 200 1 row(s) fetched - 0.002s, on 2026-04-24 at 19:37:57

Database Navigator: Filter catalogs, databases or schema

- mysql localhost:3306
 - Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_System
 - Tables
 - Views
 - vw_available_books_by_category
 - vw_customer_order_summary
 - vw_revenue_by_category
 - Indexes
 - Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
 - Triggers
 - Events
 - Pharmacy_n10004688
 - retail_analysis
 - Users
 - Administer
 - System Info

SQL Editor: <mysql> customer_shopping_behavior.sql

```

END;

-- Testing
-- Check an existing order
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
ORDER BY OrderID DESC;

-- Run the procedure
CALL sp_process_payment(31, 'Credit Card', 'Completed');

-- Check if payment was inserted
SELECT *
FROM Payments
ORDER BY PaymentID DESC
LIMIT 1;

-- Check if order status changed
SELECT OrderID, TotalAmount, OrderStatus
FROM Orders
WHERE OrderID = 31;
  
```

orders 1 X

SELECT OrderID, TotalAmount, OrderStatus

	123 OrderID	123 TotalAmount	AZ OrderStatus
1	31	35.98	Paid

Refresh Save Cancel Export data 200 1 row(s) fetched - 0.001s, on 2026-04-24 at 19:38:02

4.6 Event Verified:

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays a tree view of the server structure, including connections, databases, and users. The central pane shows a script titled "Script-11" with the following SQL code:

```
FROM Orders
WHERE OrderID = 31;

SELECT * FROM Orders ORDER BY OrderID DESC;

-- =====
-- Events

-- 4d.1 evt_cancel_unpaid_orders - Purpose & Schedule
-- Purpose:
-- Automatically cancel/remove unpaid orders that remain
-- Pending for more than 48 hours.
-- Restore stock before deleting the order.
-- Runs every 1 hour.

SET GLOBAL event_scheduler = ON;

DROP EVENT IF EXISTS evt_cancel_unpaid_orders;

CREATE EVENT evt_cancel_unpaid_orders
ON SCHEDULE EVERY 1 HOUR
DO
DELETE FROM Orders
WHERE OrderStatus = 'Pending'
AND OrderDateTime <= NOW() - INTERVAL 48 HOUR;
```

The bottom pane shows the "Statistics 1" tab with the following output:

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Sat Apr 25 08:55:12 EDT 2026
Finish time	Sat Apr 25 08:55:12 EDT 2026
Query	CREATE EVENT evt_cancel_unpaid_orders ON SCHEDULE EVERY 1 HOUR DO DELETE FROM Orders WHERE OrderStatus = 'Pending' AND OrderDateTime <= NOW() - INTERVAL 48 HOUR

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays a tree view of the server structure, including connections, databases, and users. The central pane shows a script titled "Script-11" with the following SQL code:

```
CREATE EVENT evt_cancel_unpaid_orders
ON SCHEDULE EVERY 1 HOUR
DO
DELETE FROM Orders
WHERE OrderStatus = 'Pending'
AND OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 1: Restore stock
UPDATE Books b
JOIN OrderDetails od ON b.BookID = od.BookID
JOIN Orders o ON od.OrderID = o.OrderID
SET b.Stock = b.Stock + od.Quantity
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 2: Mark restored
UPDATE Orders
SET InventoryRestored = TRUE
WHERE OrderStatus = 'Pending'
AND OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 3: Delete Payments
DELETE p
FROM Payments p
JOIN Orders o ON p.OrderID = o.OrderID
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 4: Delete OrderDetails
DELETE od
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
```

The bottom pane shows the "Statistics 1" tab with the following output:

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Sat Apr 25 08:55:29 EDT 2026
Finish time	Sat Apr 25 08:55:29 EDT 2026
Query	-- Step 1: Restore stock UPDATE Books b JOIN OrderDetails od ON b.BookID = od.BookID JOIN Orders o ON od.OrderID = o.OrderID SET b.Stock = b.Stock + od.Quantity WHERE o.OrderStatus = 'Pending' AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR

Projects X

- General
 - Connections
 - DatabaseMidterm localhost:3306
 - localhost localhost:3306
 - localhost 2 localhost:3306
 - localhost 3 localhost:3306
 - localhost 4 localhost:3306
 - Databases
 - Online_Bookstore_Management_System
 - Pharmacy_N01779257
 - SinghEyeClinic
 - SinghRetail
 - sys
 - Users
 - Administer
 - System Info
 - MySQL localhost:3306
 - MySQL 2 localhost:3306
 - MySQL 3 localhost:3306
 - MySQL 4 localhost:3306
 - Bookmarks
 - Dashboards
 - Diagrams
 - Scripts

*localhost 4> Script-11 X

```

-- Step 3: Delete Payments
DELETE p
FROM Payments p
JOIN Orders o ON p.OrderID = o.OrderID
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 4: Delete OrderDetails
DELETE od
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 5: Delete Orders
DELETE FROM Orders
WHERE OrderStatus = 'Pending'
AND OrderDateTime <= NOW() - INTERVAL 48 HOUR;

END;

-- User Roles and Permissions

CREATE USER 'bookstore_admin'@'localhost' IDENTIFIED BY 'Admin@Secure123';
CREATE USER 'bookstore_seller'@'localhost' IDENTIFIED BY 'Seller@Secure456';
CREATE USER 'bookstore_customer'@'localhost' IDENTIFIED BY 'Customer@Secure789';

```

Statistics 1 X Output

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Sat Apr 25 08:56:15 EDT 2026
Finish time	Sat Apr 25 08:56:15 EDT 2026
Query	-- Step 5: Delete Orders

EDT en Writable Smart Insert 822 : 5 [29]

Projects X

- General
 - Connections
 - DatabaseMidterm localhost:3306
 - localhost localhost:3306
 - localhost 2 localhost:3306
 - localhost 3 localhost:3306
 - localhost 4 localhost:3306
 - Databases
 - Online_Bookstore_Management_System
 - Pharmacy_N01779257
 - SinghEyeClinic
 - SinghRetail
 - sys
 - Users
 - Administer
 - System Info
 - MySQL localhost:3306
 - MySQL 2 localhost:3306
 - MySQL 3 localhost:3306
 - MySQL 4 localhost:3306
 - Bookmarks
 - Dashboards
 - Diagrams
 - Scripts

*localhost 4> Script-11 X

```

AND OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 1: Restore stock
UPDATE Books b
JOIN OrderDetails od ON b.BookID = od.BookID
JOIN Orders o ON od.OrderID = o.OrderID
SET b.Stock = b.Stock + od.Quantity
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 2: Mark restored
UPDATE Orders
SET InventoryRestored = TRUE
WHERE OrderStatus = 'Pending'
AND OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 3: Delete Payments
DELETE p
FROM Payments p
JOIN Orders o ON p.OrderID = o.OrderID
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 4: Delete OrderDetails
DELETE od
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
WHERE o.OrderStatus = 'Pending'
AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR;

-- Step 5: Delete Orders

```

Statistics 1 X Output

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Sat Apr 25 08:55:53 EDT 2026
Finish time	Sat Apr 25 08:55:53 EDT 2026
Query	-- Step 3: Delete Payments DELETE p FROM Payments p JOIN Orders o ON p.OrderID = o.OrderID WHERE o.OrderStatus = 'Pending' AND o.OrderDateTime <= NOW() - INTERVAL 48 HOUR

EDT en Writable Smart Insert 809 : 5 : 29275

4.7 User Roles and Grants Verified:

The screenshot shows the MySQL Workbench interface with the 'Online_Bookstore_Management_System' database selected. The left sidebar displays the database structure, including tables, views, indexes, procedures, triggers, and events. The main editor window contains the following SQL script:

```
ON Online_Bookstore_Management_System.*
TO 'bookstore_seller'@'localhost';

-- Customer: limited read on books and categories only

GRANT SELECT (BookID, Title, Price, Stock, CategoryID)
ON Online_Bookstore_Management_System.Books
TO 'bookstore_customer'@'localhost';

GRANT SELECT
ON Online_Bookstore_Management_System.Categories
TO 'bookstore_customer'@'localhost';

FLUSH PRIVILEGES;

-- Verification of User and Permissions

SELECT User, Host FROM mysql.user WHERE User LIKE 'bookstore_%';
SHOW GRANTS FOR 'bookstore_admin'@'localhost';
SHOW GRANTS FOR 'bookstore_seller'@'localhost';
SHOW GRANTS FOR 'bookstore_customer'@'localhost';
```

Below the SQL editor, the 'user1' tab shows the results of the verification queries in a table format:

	User	Host
1	bookstore_admin	localhost
2	bookstore_customer	localhost
3	bookstore_seller	localhost

The screenshot shows the MySQL Workbench interface with the 'Online_Bookstore_Management_System' database selected. The main editor window contains the same SQL script as the previous screenshot, but with the following queries highlighted:

```
SELECT User, Host FROM mysql.user WHERE User LIKE 'bookstore_%';
SHOW GRANTS FOR 'bookstore_admin'@'localhost';
SHOW GRANTS FOR 'bookstore_seller'@'localhost';
SHOW GRANTS FOR 'bookstore_customer'@'localhost';
```

Below the SQL editor, the 'Results 1' tab shows the results of the 'SHOW GRANTS' query for 'bookstore_admin'@'localhost' in a table format:

	Grants for bookstore_admin@localhost
1	GRANT USAGE ON *.* TO 'bookstore_admin'@'localhost';
2	GRANT ALL PRIVILEGES ON 'online_bookstore_management_system'.* TO 'bookstore_admin'@'localhost' WITH GRANT OPTION

Database Navigator: Filter catalogs, databases or scli
mysql localhost:3306

- Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_Sy...
- Tables
 - vw_available_books_by_catego...
 - vw_customer_order_summary
 - vw_revenue_by_category
- Indexes
- Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
- Triggers
- Events
- Pharmacy_n10004688
- retail_analysis
- Users
- Administer
- System Info

```

ON Online_Bookstore_Management_System.*
TO 'bookstore_seller'@'localhost';

-- Customer: limited read on books and categories only

GRANT SELECT (BookID, Title, Price, Stock, CategoryID)
ON Online_Bookstore_Management_System.Books
TO 'bookstore_customer'@'localhost';

GRANT SELECT
ON Online_Bookstore_Management_System.Categories
TO 'bookstore_customer'@'localhost';

FLUSH PRIVILEGES;

-- Verification of User and Permissions

SELECT User, Host FROM mysql.user WHERE User LIKE 'bookstore_%';

SHOW GRANTS FOR 'bookstore_admin'@'localhost';
SHOW GRANTS FOR 'bookstore_seller'@'localhost';
SHOW GRANTS FOR 'bookstore_customer'@'localhost';

```

Results 1 X

SHOW GRANTS FOR 'bookstore_seller'@'localhost' Enter a SQL expression to filter results (use Ctrl+Space)

1	2
GRANT USAGE ON ** TO 'bookstore_seller'@'localhost'	
GRANT SELECT ON 'online_bookstore_management_system'.*	TO 'bookstore_seller'@'localhost'

Refresh Save Cancel Export data 200 2

EDT en Writable Smart Insert 827 : 1 : 29305

Database Navigator: Filter catalogs, databases or scli
mysql localhost:3306

- Databases
 - class_work
 - customer_shopping_behavior_db
 - get_analysis
 - instagram_analysis_db
 - maven_marketing_data
 - Online_Bookstore_Management_Sy...
- Tables
 - vw_available_books_by_catego...
 - vw_customer_order_summary
 - vw_revenue_by_category
- Indexes
- Procedures
 - fn_discounted_price
 - fn_get_discount_rate
 - sp_place_order
 - sp_process_payment
 - sp_update_inventory
- Triggers
- Events
- Pharmacy_n10004688
- retail_analysis
- Users
- Administer
- System Info

```

ON Online_Bookstore_Management_System.*
TO 'bookstore_seller'@'localhost';

-- Customer: limited read on books and categories only

GRANT SELECT (BookID, Title, Price, Stock, CategoryID)
ON Online_Bookstore_Management_System.Books
TO 'bookstore_customer'@'localhost';

GRANT SELECT
ON Online_Bookstore_Management_System.Categories
TO 'bookstore_customer'@'localhost';

FLUSH PRIVILEGES;

-- Verification of User and Permissions

SELECT User, Host FROM mysql.user WHERE User LIKE 'bookstore_%';

SHOW GRANTS FOR 'bookstore_admin'@'localhost';
SHOW GRANTS FOR 'bookstore_seller'@'localhost';
SHOW GRANTS FOR 'bookstore_customer'@'localhost';

```

Results 1 X

SHOW GRANTS FOR 'bookstore_seller'@'localhost' Enter a SQL expression to filter results (use Ctrl+Space)

1	2
GRANT USAGE ON ** TO 'bookstore_seller'@'localhost'	
GRANT SELECT ON 'online_bookstore_management_system'.*	TO 'bookstore_seller'@'localhost'

Refresh Save Cancel Export data 200 2

EDT en Writable Smart Insert 829 : 1 : 29355